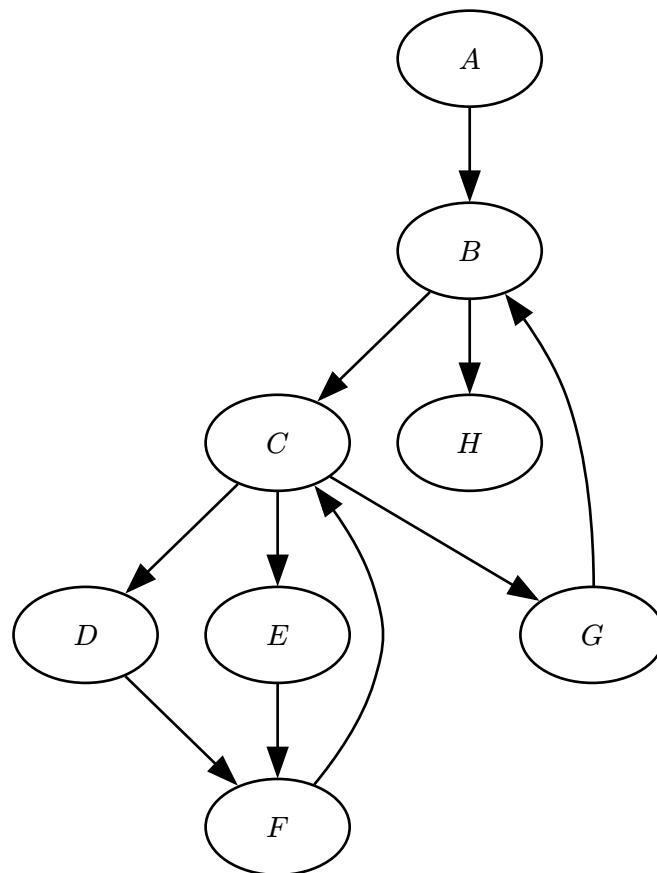


6.1

1.



2.

A, B, C, D, F, E, G, H

3.

F, D, E, G, C, H, B, A

4.

A, B, H, C, G, E, D, F

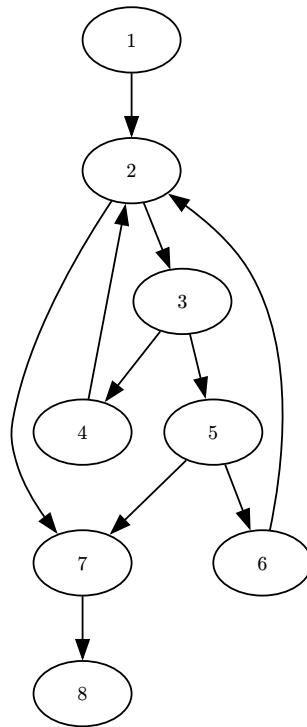
5.

```
// n is not const anymore because we need some way of getting the times out
int dfs(std::unordered_set<const Node *> &visited, Node *n, int time) {
    visited.insert(n);
    n->time_pre = time;
    time++;
    for (auto m : n->succs)
        if (visited.find(m) == visited.end())
            time = dfs(visited, m, time);
    n->time_post = time;
    time++;
    return time;
}
```

6.2

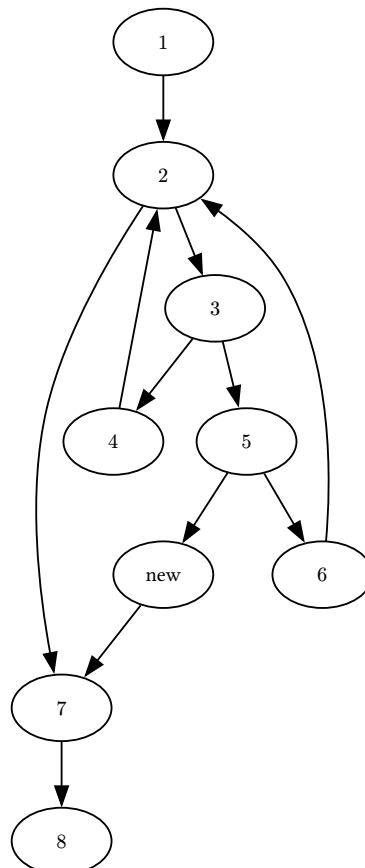
1. CFG

```
❶ A;  
❷ while (B && C) {  
  ❸ if (D || E) {  
    ❹ F;  
    continue;  
  }  
  ❺ G;  
  if (H || I) break;  
  ❻ J;  
}  
❽ K; ❿
```

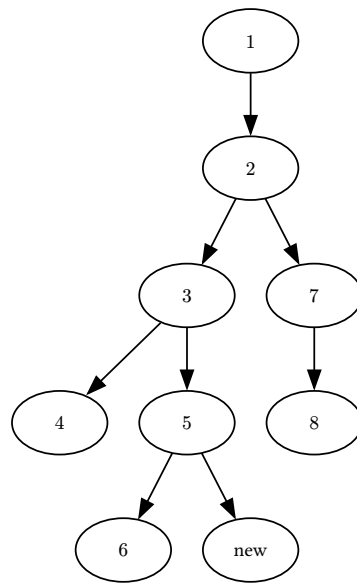


2. Critical Edges

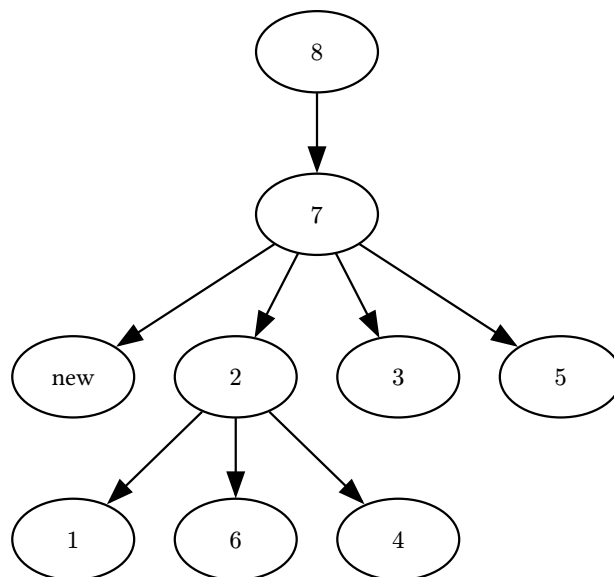
The only critical edge is from 5 to 7. We eliminate it by inserting an extra node:



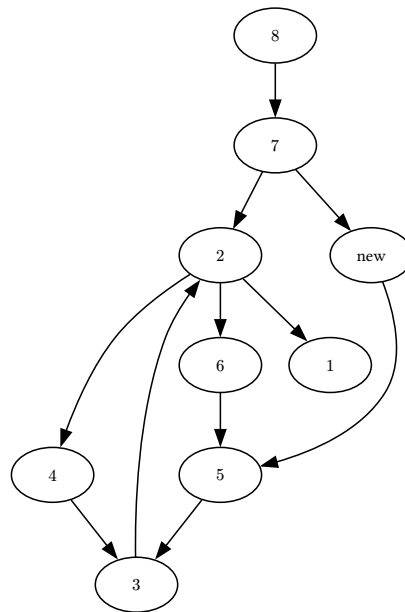
3. Dominance Tree



4. Postdominance Tree

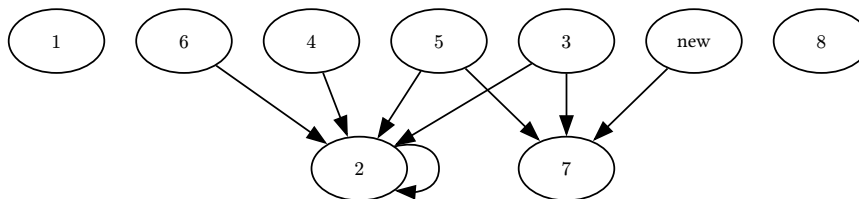


5. Reverse CFG

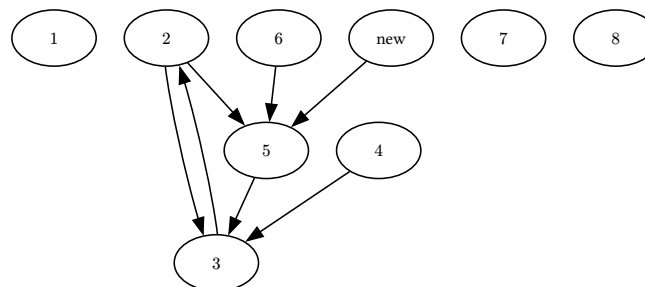


6. Dominance Frontier Graph

Of the original CFG and not the reverse one i assume? I'm guessing that this is the graph that has an edge $u \rightarrow v$ if v is in the dominance frontier of u .



6. Postdominance Frontier Graph



6.3

Dominance is reflexive: Because *all* paths to v contain v , v always dominates itself.

Dominance is transitive: When $u \succeq v$ and $v \succeq w$, then all paths from r to w include v and all paths from r to v include u . So all paths from r to w must also include u somewhere before they reach v .

Dominance is anti-symmetric: Assume that $u \succeq v$ and $v \succeq u$ (and $u \neq v$). Because of $v \succeq u$ every path from r to u contains v . Choose any of those paths and take the subpath from r up to the first occurrence of v . Because of $u \succeq v$, this subpath contains u . But then the subpath from r up to that occurrence of u does not contain v (since we picked the first occurrence of v earlier), which contradicts the assumption that $v \succeq u$.